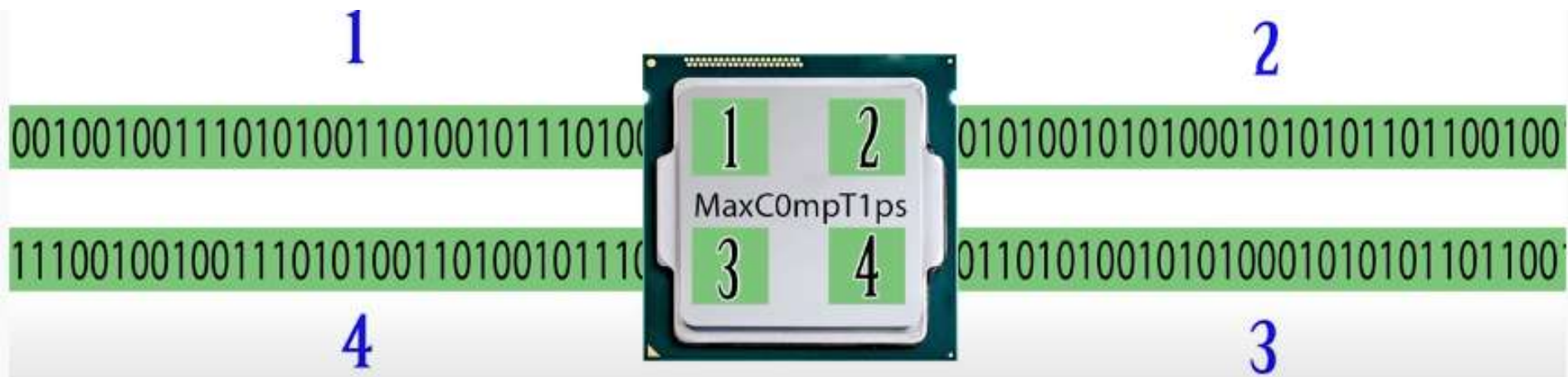


Thread and Core

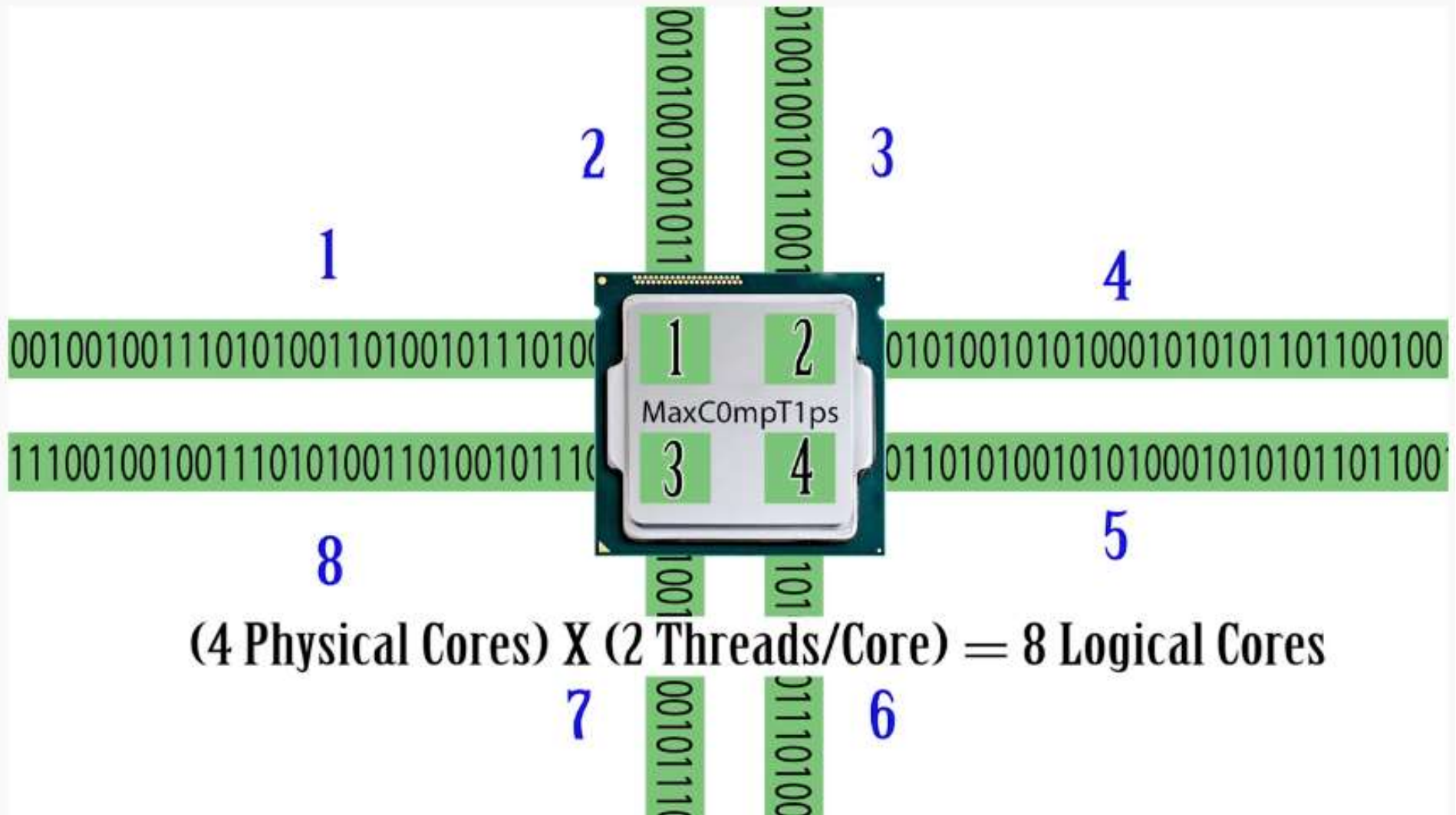
Cores Vs Threads







(4 Physical Cores) X (1 Thread/Core) = 4 Logical Cores

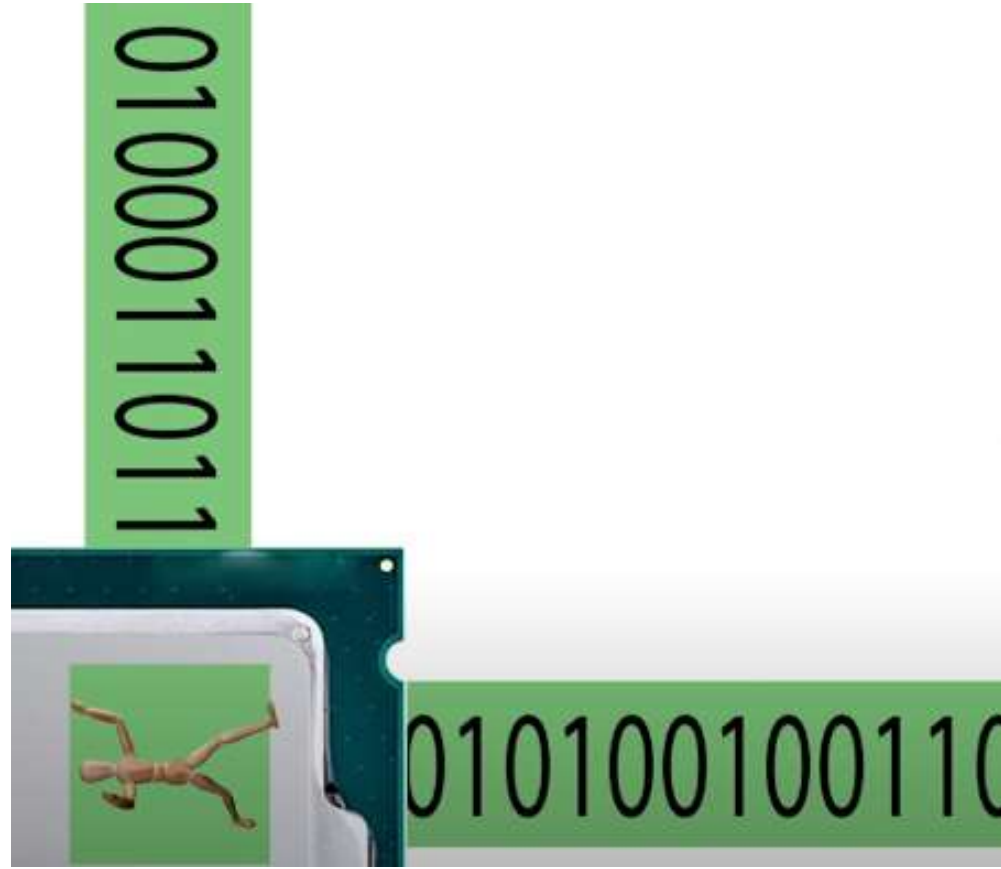


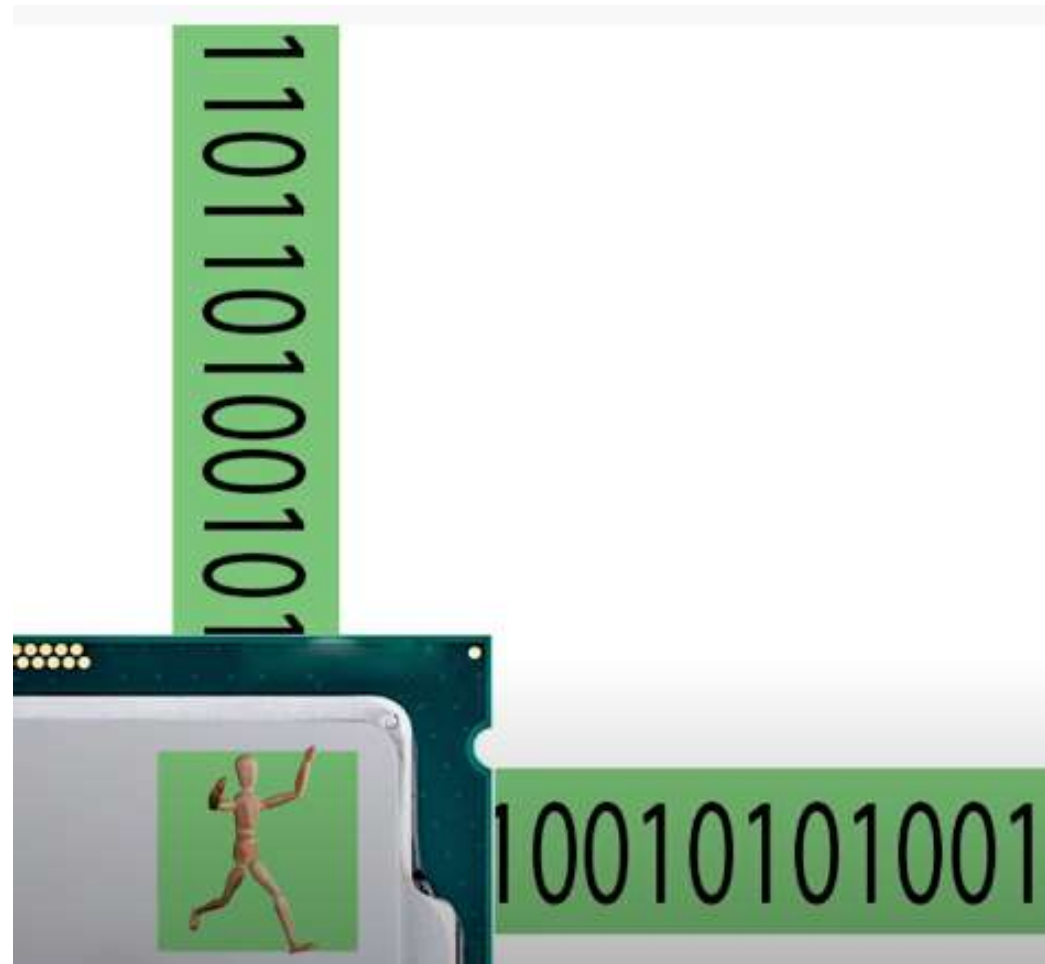
Threads > Cores

010110100011



00101101010010010111010101





Parallel Operations

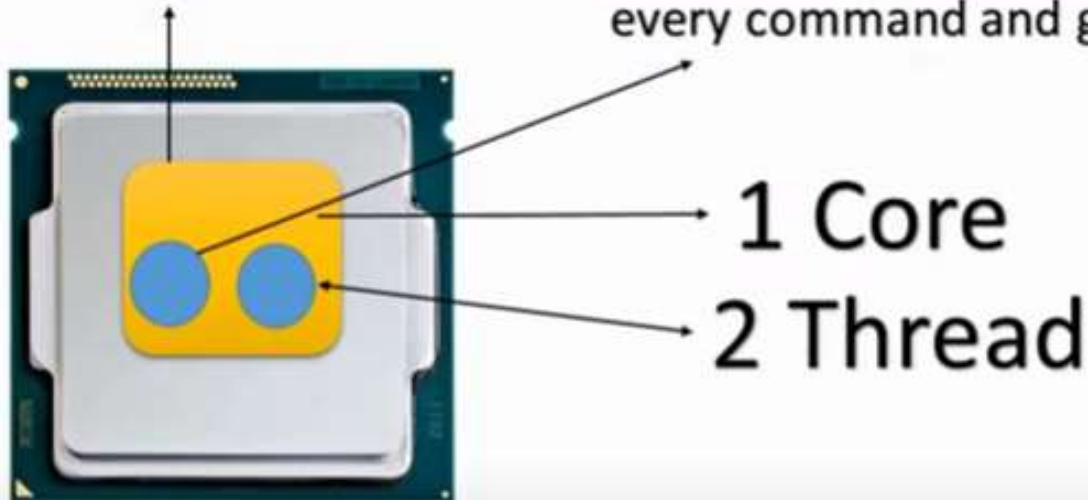
(Core to core, all operate at the same time)

Concurrent Operations

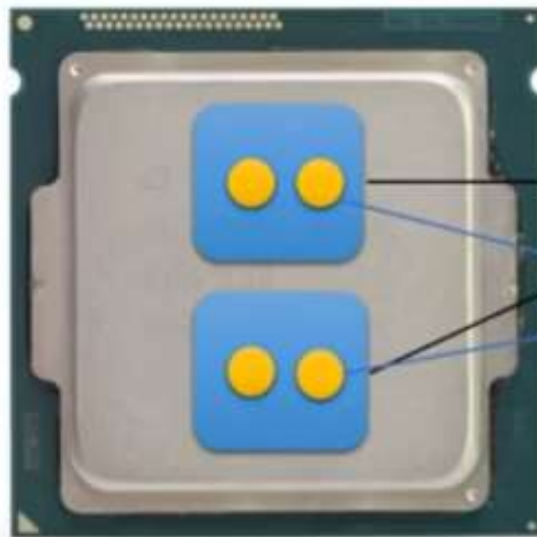
(Core to thread, operate by switching)

Core is a physical part of CPU

Thread is logical part of core that executes every command and gives output.



One CPU core can handle two separate tasks at once using two logical threads.



2 Cores
(4 Threads)

Process vs Threads

2

□ Similarities

- Like processes threads share CPU and only one thread active (running) at a time.
- Like processes, threads within a processes, threads within a processes execute sequentially.
- Like processes, thread can create children.
- And like process, if one thread is blocked, another thread can run.

□ Differences

- Unlike processes, threads are not independent of one another.
- Unlike processes, all threads can access every address in the task .
- Unlike processes, thread are design to assist one other.
- As processes may originate from different users they may or may not assist one another.

But Why Threads?

3

- ❑ A process with multiple threads make a great server (for example printer server.)
- ❑ Because threads can share common data, they do not need to use inter process communication.
- ❑ Because of the very nature, threads can take advantage of multiprocessors.
- ❑ Threads only need a stack and storage for registers therefore, threads are easy to create.
- ❑ Threads use very little resources of an operating system in which they are working.
- ❑ Threads do not need new address space, global data, program code or operating system resources.
- ❑ Context switching are fast when working with threads. The reason is that we only have to save and/or restore PC, SP and registers.

Advantages of Threads over Multiple Processes

4

- Context Switching : Threads are very inexpensive to create and destroy, and they are inexpensive to represent.
 - For example, they require space to store, the PC, the SP, and the general-purpose registers, but they do not require space to share memory information, Information about open files of I/O devices in use, etc.
 - With so little context, it is much faster to switch between threads. In other words, it is relatively easier for a context switch using threads.
- Sharing : Threads allow the sharing of a lot resources that cannot be shared in process, for example, sharing code section, data section, Operating System resources like open file etc.

Disadvantages of Threads over Multiple Processes

6

- ❑ Blocking: If the kernel is single threaded, a system call of one thread will block the whole process and CPU may be idle during the blocking period.
- ❑ Security Since there is, an extensive sharing among threads there is a potential problem of security.
 - ❑ Possible that one thread over writes the stack of another thread (or damaged shared data) although it is very unlikely since threads are meant to cooperate on a single task.

Thread Libraries

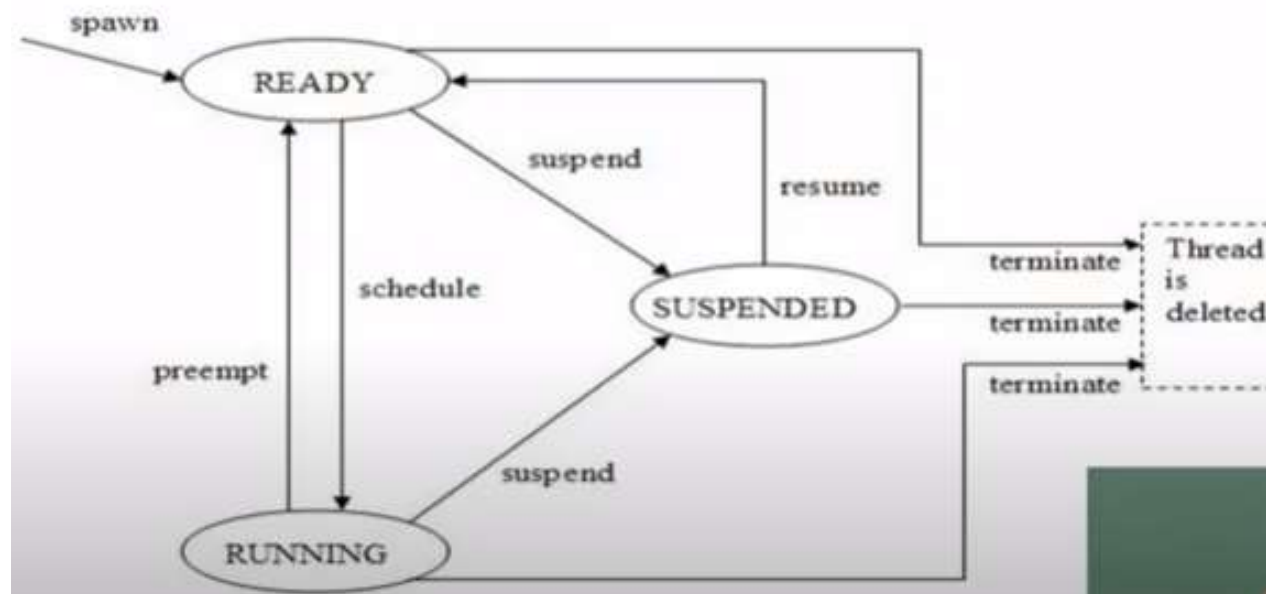
9

- Thread library provides programmer with API for creating and managing threads.
- Two primary ways of implementing
 - ▣ Library entirely in user space
 - ▣ Kernel-level library supported by the OS

Thread States

- Primary states:
 - ▣ Running, Ready and Blocked.
- Operations to change state:
 - ▣ **Spawn**: new thread provided register context and stack pointer.
 - ▣ **Block**: event wait, save user registers, PC and stack pointer
 - ▣ **Unblock**: moved to ready state
 - ▣ **Finish**: deallocate register context and stacks.

Thread State Diagram

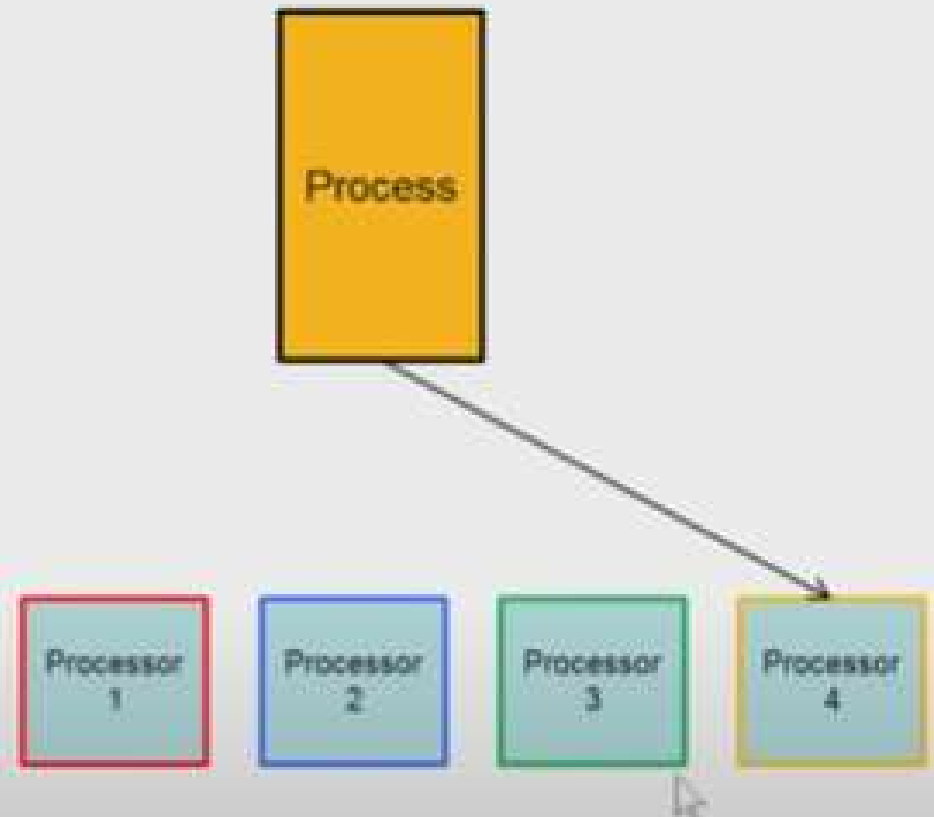


```
#include <stdio.h>

unsigned long addall(){
    int i=0;
    unsigned long sum=0;

    while (i< 100000000){
        sum += i;
        i++;
    }
    return sum;
}

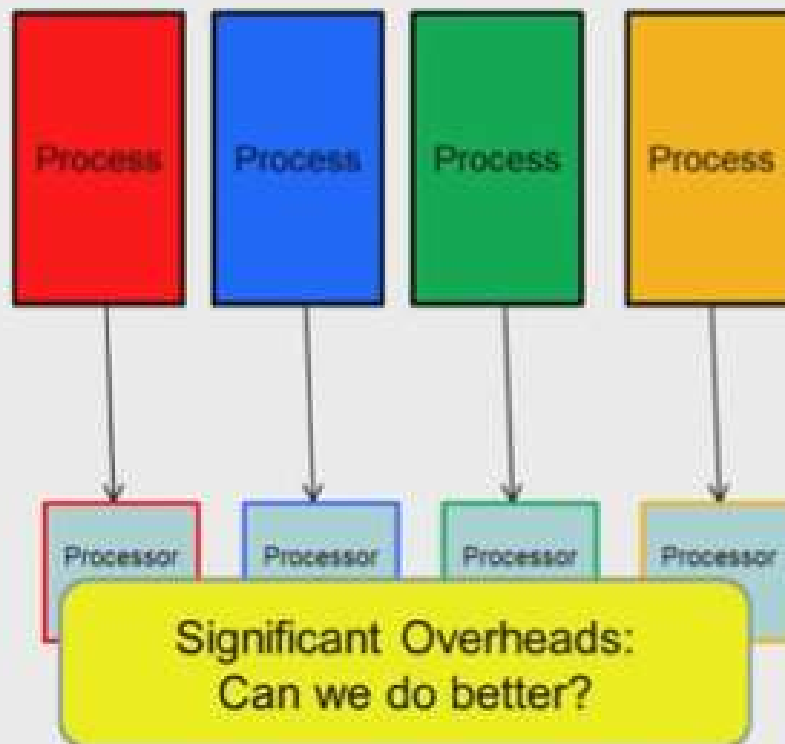
int main()
{
    unsigned long sum;
    srand(time(NULL));
    sum = addall();
    printf("%lu\n", sum);
}
```



Speeding up with multiple processes

$$10000000 / 4 = 2500000$$

Create 4 processes, each loop does 1/4th of the work



Properties:

4 fork system calls needed; one for creating each process

Each process is isolated from each other

IPC mechanisms to communicate – more system calls

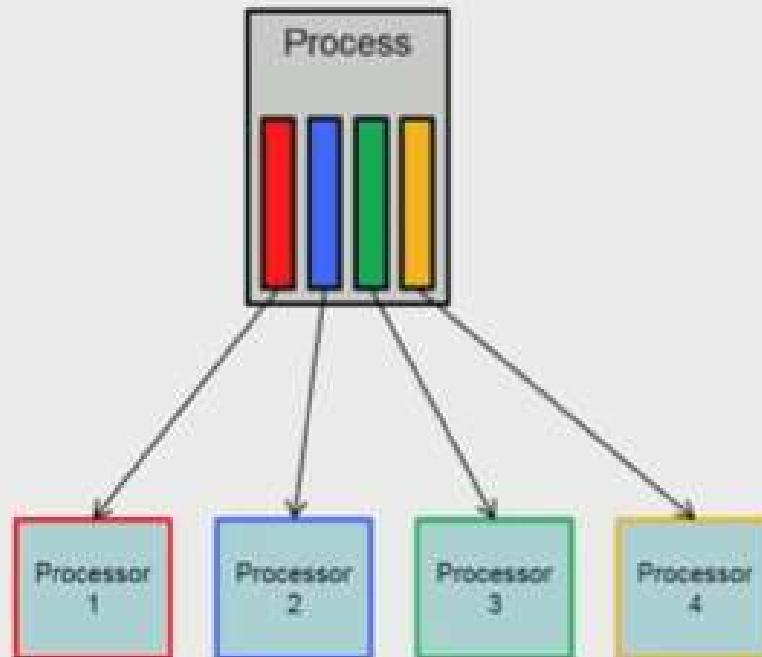
Process management with system calls

Each process has its own memory map – its own instructions, data, stack, heap, etc.

Thread model

$$10000000 / 4 = 2500000$$

Create 1 process with 4 threads; each loop does 1/4th of the work



Properties:

1 fork system call needed;
4 threads need to be
created --- much more
lighter.

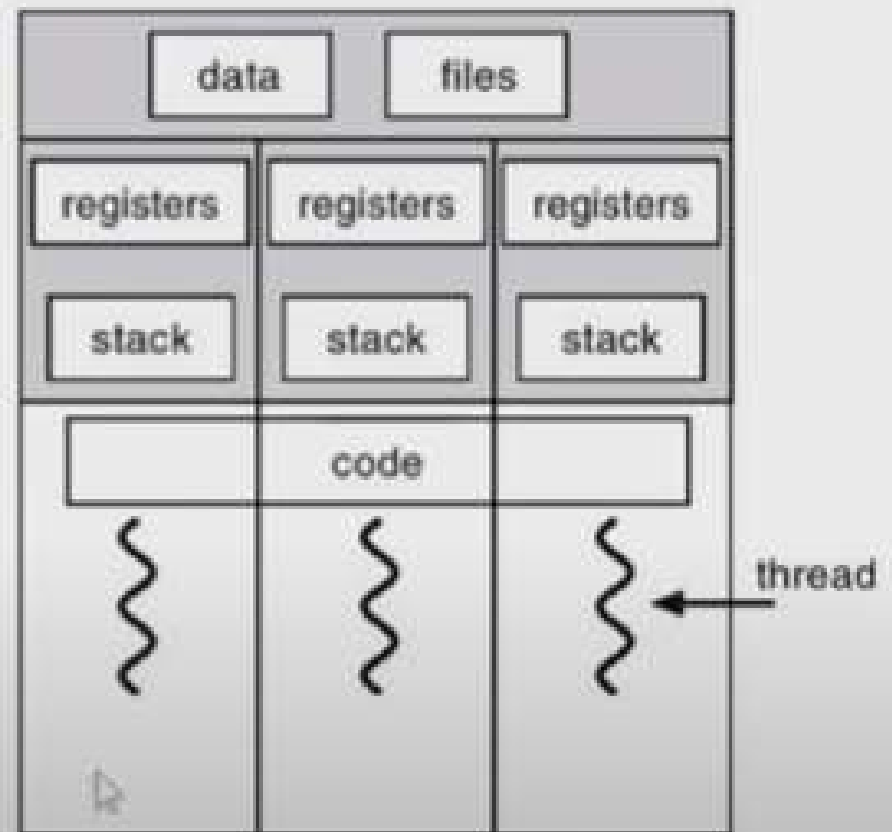
Each thread is not isolated
from others

Management of threads
with fewer or no system
calls.

Shared instructions, global,
and heap regions, each
thread has its own stack

Threads

- Separate streams of execution within a single process
- Threads in a process not isolated from each other
- Each thread state (thread control block) contains
 - Registers (including EIP, ESP)
 - stack



Threads vs Processes

- A thread has no data segment or heap
- A thread cannot live on its own. It needs to be attached to a process
- There can be more than one thread in a process. Each thread has its own stack
- If a thread dies, its stack is reclaimed
- A process has code, heap, stack, other segments
- A process has at-least one thread.
- Threads within a process share the same code, files.
- If a process dies, all threads die.

pthread library

- Create a thread in a process

```
int pthread_create(pthread_t *thread,  
                  const pthread_attr_t *attr,  
                  void *(*start_routine) (void *),  
                  void *arg);
```

Thread identifier (TID) much like

Pointer to a function,
which starts execution in a
different thread

Arguments to the function

- Destroying a thread

```
void pthread_exit(void *retval);
```

pthread library contd.

- Join : Wait for a specific thread to complete

```
int pthread_join(pthread_t thread, void **retval);
```

TID of the thread to wait for



Exit status of the thread



Note. You need to link the pthread library



```
$ gcc threads.c -lpthread  
$ ./a.out
```

Who manages threads?

- Two strategies
 - User threads
 - Thread management done by user level thread library. Kernel knows nothing about the threads
 - Kernel threads
 - Threads directly supported by the kernel.
 - Known as light weight processes.

Threading Issues (Part-1)

The fork() and exec() System Calls

The semantics of the fork() and exec() system calls change in a multithreaded program.

Issue

If one thread in a program calls fork(), does the new process duplicate all threads, or is the new process single-threaded?

Solution

Some UNIX systems have chosen to have two versions of fork (), one that duplicates all threads and another that duplicates only the thread that invoked the fork () system call



But which version of fork () to use and when ?

Also, if a thread invokes the exec () system call, the program specified in the parameter to exec () will replace the entire process



But which version of `fork()` to use and when ?

Also, if a thread invokes the `exec()` system call, the program specified in the parameter to `exec()` will replace the entire process—including all threads.

Which of the two versions of `fork()` to use depends on the application.

If `exec()` is called immediately after forking

Then duplicating all threads is unnecessary, as the program specified in the parameters to `exec()` will replace the process.

In this instance, duplicating only the calling thread is appropriate.

If the separate process does not call `exec()` after forking

Then the separate process should duplicate all threads.

Threading Issues (Part-2)

Thread Cancellation

Thread cancellation is the task of terminating a thread before it has completed.



If multiple threads are concurrently searching through a database and one thread returns the result, the remaining threads might be canceled.



When a user presses a button on a web browser that stops a web page from loading any further, all threads loading the page are canceled.

A thread that is to be canceled is often referred to as the **target thread**.



Cancellation of a target thread may occur in two different scenarios:

1. **Asynchronous cancellation:** One thread immediately terminates the target thread.
2. **Deferred cancellation:** The target thread periodically checks whether it should terminate, allowing it an opportunity to terminate itself in an orderly fashion.

Where the difficulty with cancellation lies:

In situations where:

- Resources have been allocated to a canceled thread
- A thread is canceled while in the midst of updating data it is sharing with other threads.

Often, the OS will reclaim system resources from a canceled thread but will not reclaim all resources.

Therefore, canceling a thread asynchronously may not free a necessary system-wide resource.

Often, the OS will reclaim system resources from a canceled thread but will not reclaim all resources.

Therefore, canceling a thread asynchronously may not free a necessary system-wide resource.

With deferred cancellation:

One thread indicates that a target thread is to be canceled.

But cancellation occurs only after the target thread has checked a flag to determine if it should be canceled or not.

This allows a thread to check whether it should be canceled at a point when it can be canceled safely.